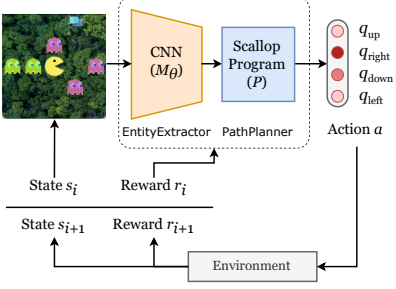




(a) Three states of one gameplay session.



(b) Architecture of application with Scallop.

```

1 class PacManAgent(torch.nn.Module):
2     def __init__(self, grid_dim, cell_size):
3         # initializations...
4         self.extract_entities =
5             EntityExtractor(grid_dim, cell_size)
6         self.path_planner = ScallopModule(
7             file="path_planner.scl",
8             provenance="diff-top-k-proofs", k=1,
9             input_mappings={"actor": cells,
10                            "goal": cells, "enemy": cells},
11             output_mappings={"next_action": actions})
12
13     def forward(self, game_state_image):
14         actor, goal, enemy =
15             self.extract_entities(game_state_image)
16         next_action = self.path_planner(
17             actor=actor, goal=goal, enemy=enemy)
18         return next_action

```

(c) Snippet of implementation in Python.

Fig. 2. Illustration of a planning application PacMan-Maze in Scallop.

the agent: 1 upon reaching the goal, and 0 otherwise. This procedure repeats until the game ends or reaches a predefined limit on the number of steps.

A popular RL method to realize our application is Q -Learning. Its goal is to learn a function $Q : S \times A \rightarrow \mathbb{R}$ that returns the expected reward of taking action a_i in state s_i .¹ Since the game states are images, we employ Deep Q -Learning [Mnih et al. 2015], which approximates the Q function using a convolutional neural network (CNN) model with learned parameter θ . An end-to-end deep learning based approach for our application involves training the model to predict the Q -value of each action for a given game state. This approach takes 50K training episodes to achieve a 84.9% test success rate, where a single episode is one gameplay session from start to end.

In contrast, a neurosymbolic solution using Scallop only needs 50 training episodes to attain a 99.4% test success rate. Scallop enables to realize these benefits of the neurosymbolic paradigm by decomposing the agent’s task into separate neural and symbolic components, as shown in Fig. 2b. These components perform sub-tasks that are ideally suited for their respective paradigms: the neural component perceives pixels of individual cells of the image at each step to identify the entities in them, while the symbolic component reasons about enemy-free paths from the actor to the goal to determine the optimal next action. Fig. 2c shows an outline of this architecture’s implementation using the popular PyTorch framework.

Concretely, the neural component is still a CNN, but it now takes the pixels of a single cell in the input image at a time, and classifies the entity in it. The implementation of the neural component (EntityExtractor) is standard and elided for brevity. It is invoked on lines 14-15 with input `game_state_image`, a tensor in $\mathbb{R}^{200 \times 200 \times 3}$, and returns three $\mathbb{R}^{5 \times 5}$ tensors of entities. For example, `actor` is an $\mathbb{R}^{5 \times 5}$ tensor and `actorij` is the probability of the actor being in cell (i, j) . A representation of the entities is then passed to the symbolic component on lines 16-17, which derives the Q -value of each action. The symbolic component, which is configured on lines 6-11,

¹We elide the Q -Learning algorithm as it is not needed to illustrate the neurosymbolic programming aspects of our example.

```

1 // File path_planner.scl
2 type actor(x: i32, y: i32), goal(x: i32, y: i32), enemy(x: i32, y: i32)
3
4 const UP = 0, DOWN = 1, RIGHT = 2, LEFT = 3
5 rel safe_cell(x, y) = range(0, 5, x), range(0, 5, y), not enemy(x, y)
6 rel edge(x, y, x, yp, UP) = safe_cell(x, y), safe_cell(x, yp), yp == y + 1
7 // Rules for DOWN, RIGHT, and LEFT edges are omitted...
8
9 rel next_pos(p, q, a) = actor(x, y), edge(x, y, p, q, a)
10 rel path(x, y, x, y) = next_pos(x, y, _)
11 rel path(x1, y1, x3, y3) = path(x1, y1, x2, y2), edge(x2, y2, x3, y3, _)
12 rel next_action(a) = next_pos(p, q, a), path(p, q, r, s), goal(r, s)

```

Fig. 3. The logic program of the PacMan-Maze application in Scallop.

comprises the Scallop program shown in Fig. 3. We next illustrate the three key design decisions of Scallop outlined in Section 1 with respect to this program.

Relational Model. In Scallop, the primary data structure for representing symbols is a *relation*. In our example, the game state can be symbolically described by the kinds of entities that occur in the discrete cells of a 5×5 grid. We can therefore represent the input to the symbolic component using binary relations for the three kinds of entities: actor, goal, and enemy. For instance, the fact `actor(2,3)` indicates that the actor is in cell (2,3). Likewise, since there are four possible actions, the output of the symbolic component is represented by a unary relation `next_action`.

Symbols extracted from unstructured inputs by neural networks have associated probabilities, such as the $\mathbb{R}^{5 \times 5}$ tensor `actor` produced by the neural component in our example (line 14 of Fig. 2c). Scallop therefore allows to associate tuples with probabilities, e.g. `0.96 :: actor(2,3)`, to indicate that the actor is in cell (2,3) with probability 0.96. More generally, Scallop enables the conversion of tensors in the neural component to and from relations in the symbolic component via input-output mappings (lines 9-11 in Fig. 2c), allowing the two components to exchange information seamlessly.

Declarative Language. Another key consideration in a neurosymbolic language concerns what constructs to provide for symbolic reasoning. Scallop uses a declarative language based on Datalog, which we present in Section 3 and illustrate here using the program in Fig. 3. The program realizes the symbolic component of our example using a set of logic rules. Following Datalog syntax, they are “if-then” rules, read right to left, with commas denoting conjunction.

Recall that we wish to determine an action a (up, down, right, or left) to a cell (p, q) that is adjacent to the actor’s cell (x, y) such that there is an enemy-free path from (p, q) to the goal’s cell (r, s) . The nine depicted rules succinctly compute this sophisticated reasoning pattern by building successively complex relations, with the final rule (on line 14) computing all such actions.²

The arguably most complex concept is the path relation which is recursively defined (on lines 10-11). Recursion allows to define the pattern succinctly, enables the trained application to generalize to grids arbitrarily larger than 5×5 unlike the purely neural version, and makes the pattern more amenable to synthesis from input-output examples. Besides recursion, Scallop also supports negation and aggregation; together, these features render the language adequate for specifying common high-level reasoning patterns in practice.

Differentiable Reasoning. With the neural and symbolic components defined, the last major consideration concerns how to train the neural component using only end-to-end supervision.

²We elide showing an auxiliary relation of all grid cells tagged with probability 0.99 which serves as the penalty for taking a step. Thus, longer paths are penalized more, driving the symbolic program to prioritize moving closer to the goal.

In our example, supervision is provided in the form of a reward of 1 or 0 per gameplay session, depending upon whether or not the sequence of actions by the agent successfully led the actor to the goal without hitting any enemy. This form of supervision, called algorithmic or weak supervision, alleviates the need to label intermediate relations at the interface of the neural and symbolic components, such as the actor, goal, and enemy relations. However, this also makes it challenging to learn the gradients for the tensors of these relations, which in turn are needed to train the neural component using gradient-descent techniques.

The key insight in Scallop is to exploit the structure of the logic program to guide the gradient calculations. The best heuristic for such calculations depends on several application characteristics such as the amount of available data, reasoning patterns, and the learning setup. Scallop provides a convenient interface for the user to select from a library of built-in heuristics. Furthermore, since all of these heuristics follow the structure of the logic program, Scallop implements them uniformly as instances of a general and extensible *provenance framework*, described in Section 4. For our example, line 8 in Fig. 2c specifies `diff-top-k-proofs` with `k=1` as the heuristic to use, which is the default in Scallop that works best for many applications, as we demonstrate in Section 6.

3 LANGUAGE

We provide an overview of Scallop’s language for symbolic reasoning which we previously illustrated in the program shown in Fig. 3. Appendix A provides the formal syntax of the language. Here, we illustrate each of the key constructs using examples of inferring kinship relations.

3.1 Data Types

The fundamental data type in Scallop is set-valued relations comprising tuples of statically-typed primitive values. The primitive data types include signed and unsigned integers of various sizes (e.g. `i32`, `usize`), single- and double-precision floating point numbers (`f32`, `f64`), boolean (`bool`), character (`char`), and string (`String`). The following example declares two binary relations, `mother` and `father`:

```
type mother(c: String, m: String), father(c: String, f: String)
```

Values of relations can be specified via individual tuples or a set of tuples of constant literals:

```
rel mother("Bob", "Christine") // Christine is Bob's mother
rel father = {("Alice", "Bob"), ("John", "Bob")} // Bob is father of two kids
```

As a shorthand, primitive values can be named and used as constant variables:

```
const FATHER = 0, MOTHER = 1, GRANDMOTHER = 2, ... // other relationships
rel composition(FATHER, MOTHER, GRANDMOTHER) // father's mother is grandmother
```

Type declarations are optional since Scallop supports type inference. The type of the `composition` relation is inferred as `(usize, usize, usize)` since the default type of unsigned integers is `usize`.

3.2 (Horn) Rules

Since Scallop’s language is based on Datalog, it supports “if-then” rule-like Horn clauses. Each rule is composed of a head atom and a body, connected by the symbol `:-` or `=`. The following code shows two rules defining the `grandmother` relation. Conjunction is specified using “,”-separated atoms within the rule body whereas disjunction is specified by multiple rules with the same head predicate. Each variable appearing in the head must also appear in some positive atom in the body (we introduce negative atoms below).

```
rel grandmother(a, c) :- father(a, b), mother(b, c) // father's mother
rel grandmother(a, c) :- mother(a, b), mother(b, c) // mother's mother
```